UNIT-II

Stacks: Definition – operations - applications of stack. Queues: Definition - operations - Priority queues - De queues – Applications of queue. Linked List: Singly Linked List, Doubly Linked List, Circular Linked List, linked stacks, Linked queues, Applications of Linked List – Dynamic storage management – Generalized list.

1. List the stack operation(Nov 2018)

In Stack, we can perform two operations namely Push and Pop.

Push: Push means inserting a new element into the stack. Insertion can be done by incrementing top by 1.

Pop: Pop means deleting an element from the stack. Deletion can be done by decrementing top by 1.

2.State the Advantage of linked list over array(Nov 2018)

Arrays	Linked List
➢ Size of any arrays is fixed.	➢ Size of a list is variable.
➢ It is necessary to specify the number of elements during the declaration.	➢ It is not necessary to specify the number of elements during the declaration.
➢ It occupies less memory space than linked list for the same memory number of elements.	➢ It occupies more memory space.

3. Difference between queue and linked list(Nov 2018)

Queue	Linked List
➢ A Queue is an ordered collection of items, where all insertions are made at the end of the sequence called Rear and all deletions always are made from the beginning of the sequence called Front.	➢ Size of a list is variable.
➢ In Queue, we get data items out in same order compared to the order they have been put into the queue.	➢ It is not necessary to specify the number of elements during the declaration.
➢ Queue is otherwise called as FIFO.	➢ It occupies more memory space.

4. What is a list? (Dec 2016)

- A *generalized list*, A, is a finite sequence of $n > 0$ elements, $_1, ..., _n$ where the $_i$ are either atoms or lists.
- The elements $_i$, $1 \ i \ n$ which are not atoms are said to be the *sub lists* of A.
- The list A itself is written as $A = (_1, ..., _n)$. A is the *name* of the list $(_1, ..., _n)$ and n its *length*.
- By convention, all list names will be represented by capital letters. Lower case letters will be used to represent atoms. If n 1, then $_1$ is the *head* of A while $(_2, ..., _2)$ is the *tail* of A.

5. State the Application of stack (Dec 2016, Nov 2015))

Some of the applications of Stack are:

i. Matching of nested parenthesis in a mathematical expression.
ii. Conversion of infix to postfix.
iii. Evaluation of postfix.

6. Write the advantage of linked list

The advantages in using a linked list are

- It is not necessary to specify the number if elements in a linked list during its declaration
- Linked list can grow and shrink in size del=pending upon the insertion and deletion that occurs in the list.
- Insertions and deletions at any place in a list can be handled easily and efficiently
- A linked list does not waste any memory space.

7. Convert into reverse polish rotation (a-b)(c(d+e-f(g/h)))

a b - c d e + f g h / -

8. Write the different way to implement the List (April 2017)

There are two general-purpose List implementations — Array List and Linked List. Most of the time, you'll probably use Array List, which offers constant-time positional access and is just plain fast. It does not have to allocate a node object for each element in the List , and it can take advantage of System.

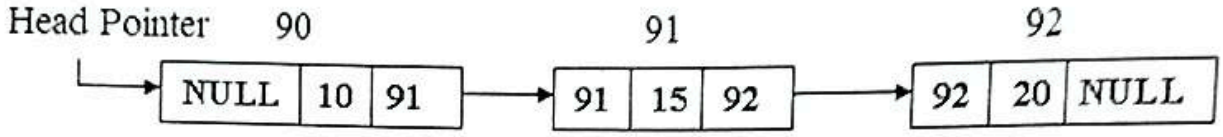
9. Evaluate the value of expression in postfix notation 752+*415-/-(Nov 2017)

The value of the above postfix conversion is -48.

10. Explain the various structures of queue (Nov 2017)

1. Enqueue
2. Dequeue

11. Draw the node structure of doubly linked list (Nov 2015)



12. What is de-queue? What are its variants?(Sep 2020)

- De queue is the short form of double ended queue
- In which we can insert or delete the elements either at front or rear
  Its variant are

1. **Input restricted Dequeue:**
2. **Output restricted dequeues**

13. Define Linear List (Sep 2020)

The elements are ordered within the linear list in a linear sequence. ... Linear lists are usually simply denoted as lists. Unlike an array, a list is a data structure allowing insertion and deletion of elements at an arbitrary position of the sequence.

11 Marks

Linked List: Singly Linked List, Doubly Linked List, Circular Linked List

1. **Explain Array and linked list representation of queue.**
   we will implement the queue using a linked list as well as with an array.

We will maintain two pointers - tail and head to represent a queue. head will always point to the oldest element which was added and tail will point where the new element is going to be added.

head and tail in queue using array

To insert any element, we add that element at tail and increase the tail by one to point to the next element of the array.

Inserting element in queue using array

Suppose tail is at the last element of the queue and there are empty blocks before head as shown in the picture given below.

Circular order in queue using array

In this case, our tail will point to the first element of the array and will follow a circular order.

Tail at 1 in circular order for queue using array

Initially, the queue will be empty i.e., both head and tail will point to the same location i.e., at index 1. We can easily check if a queue is empty or not by checking if head and tail are pointing to the same location or not at any time.

Empty queue using an array

Stacks: Definition – operations - applications of stack.

2. Explain Stack application Evaluation of expression with example(Nov 2018)(Nov 2017).

STACK APPLICATIONS

- Infix into postfix Expression.
- Expression Evaluation.

EVALUATING POSTFIX EXPRESSION:

Once the expression has been parsed into postfix form, another stack can be used to evaluate postfix expression. As an example, consider the postfix expression,

AB*CDE/-+ \0

Let us suppose that A, B, C, D and E are the symbols used associated with values,

A=5,B= 3,C=6,D=8 & E=2.

To evaluate such an expression, we repeatedly read characters from the postfix expression. If the character read is an operand, push the value associated with it onto the stack. If it is an operator, pop two values from the stack, apply the operator to them and push the result back on the stack. The process is illustrated in the following figure.

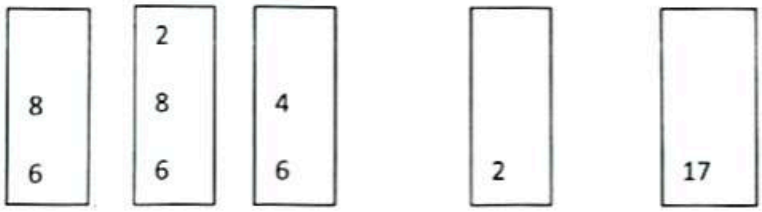
1. Read A 2.Read B 3.Read *(Pop 2 elemnts Perform opn and push res in stack) 4.Read C

5. Read D

6. Read E

7. Read / (Pop 2 elemnts Perform opn and push res in stack)

8. Read- 9. Read+ 10. Read \0

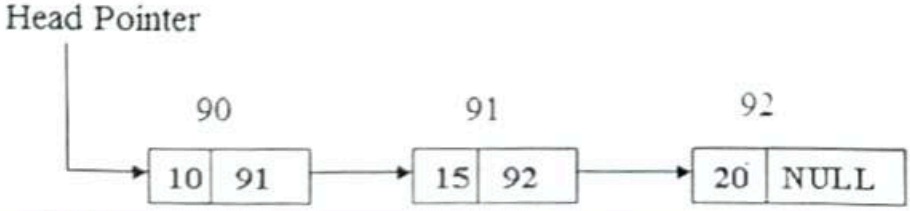


End of postfix expression.

Answer is on top of stack

3. Explain in detail about linked list.(Nov 2019,Nov 2014)

- In the single linked list each link has single link to the next node.
- It is otherwise called as linear linked list.
- It contains the head pointer which holds the address of the first node.
- Using head pointer only we can access entire linked list.
- The below diagram shows the single linked



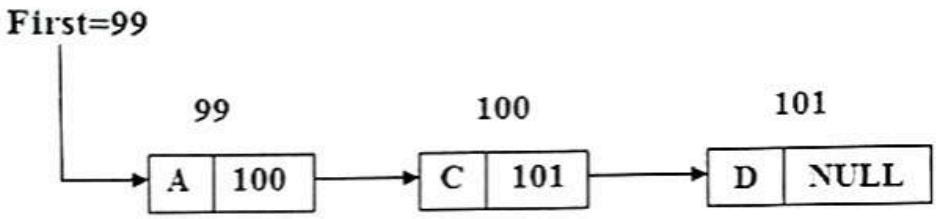
- In single linked list we can traverse in one direction from head to null.
- We cant move in reverse direction i.e. from null to head.
- Link field of last node have the null pointer indicates the end of the list.

Operations In Single Linked List:

- We can insert and delete the node in a single linked list.

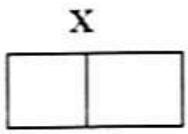
Inserting The Node Into A Single Linked List:

Consider the linked list:



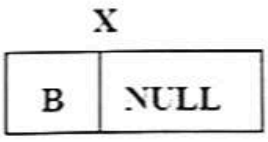
Where first is a pointer which holds the address of the first node.

If we want to insert a data B , into the list , first we need the empty node with empty data and link field.



Consider x is the address of new node.

Now put the data B into the data field of a new node and put NULL into the link field.

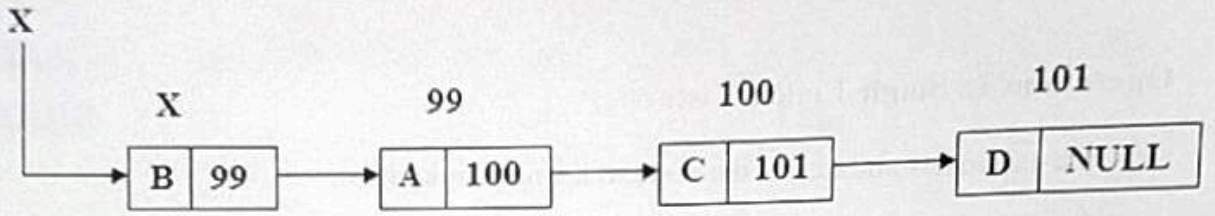


We can insert the new node by any one of the following position:

1. As a first node
2. As a intermediate node
3. As a last node.

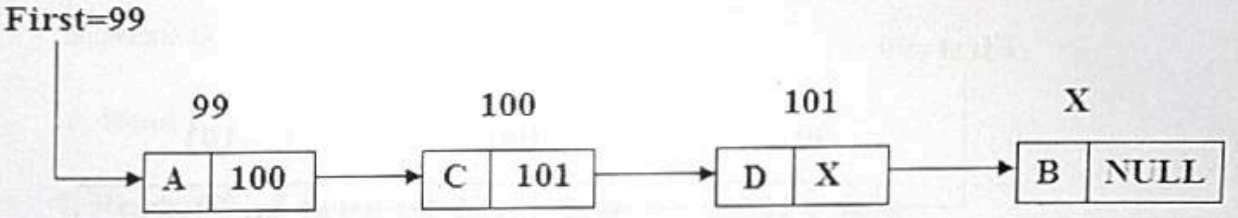
Inserting as a first node:

If we want to insert a new node as a first node, the link field of new node is replaced by FIRST and FISRT is replaced by X as shown below.



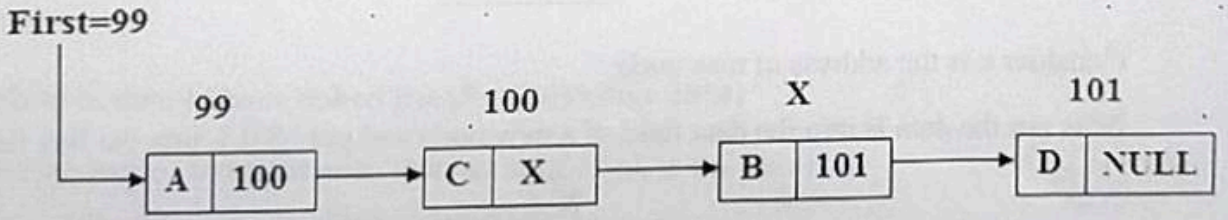
Inserting as a last node:

If we want to insert a node as a last node, the link field of new node is replaced by NULL and the link field of last but previous link field is replaced by X as shown below.



Inserting as an intermediate node:

- If we want to insert a new node as the intermediate node, first we need the address of the previous node.
- Now insert the new node in between the respective node as shown below
- If we want to insert a new node X as intermediate node, insert in between C and D.
- The link field of 99 is 100, the link field of 100 is 101 and the link field of 101 is 102.



4. Why we need Dynamic memory management (Nov 2014)

When we do not know how much amount of memory would be needed for the program beforehand.

When we want data structures without any upper limit of memory space.

When you want to use your memory space more efficiently. Example: If you have allocated memory space for a 1D array as array[20] and you end up using only 10 memory spaces then the remaining 10 memory spaces would be wasted and this wasted memory cannot even be utilized by other program variables.

Dynamically created lists insertions and deletions can be done very easily just by the manipulation of addresses whereas in case of statically allocated memory insertions and deletions lead to more movements and wastage of memory.

When you want you to use the concept of structures and linked list in programming, dynamic memory allocation is a must.

5. Explain about the push and pop operation in stack (Dec 2016)

STACK

- A stack is an ordered collection of data items.
- New data items may be inserted or deleted in a stack.
- But these insertion (PUSH) and deletion (POP) operation can be done only at one end called the top of the stack.
- Top is a pointer which always point to the top of the stack.
- The last element inserted in a stack is the first element to be removed.
- Stack is also referred to as Last in First out (LIFO) lists.

PUSH OPERATION:

- Push operation inserts an element onto the stack.
- An attempt to push an element onto the stack , when the stack is full, causes an overflow.

Push operation involves:

1. Check whether the stack is full before attempting to push an element to the stack.
2. Increment the top pointer.
3. Push the element onto the top of the stack.

ALGORITHM:

STACK - Array to hold elements

N – Total number of elements

TOP – Denotes the top element in the stack

Item – The element to be inserted at the top of a stack

1. if(TOP>=N) [Check for stack overflow]

   Then CALL STACK_FULL

   Exit

2. TOP<-TOP+1 [Increment TOP]

3. STACK [TOP]<-Item [Insert element]

   End PUSH

POP OPERATION:

- POP operation removes an element from the stack.
- An attempt to pop an element from the stack, when the array is empty, causes an underflow.

Pop operation involves:

1. Check whether the stack is empty before attempting to pop an element from the stack.
2. Decrement the top pointer.
3. Pop the element from the top of the stack.

ALGORITHM:

STACK – Array to hold elements

TOP – Denotes the top element in the stack

Item – The element to be deleted from the top of the stack

1. If (TOP<=0) [Check for underflow on stack]

   Then Call STACK_EMPTY - Exit

2. Item<-STACK [TOP] [Return top element of the stack]

3. TOP<-TOP-1 [Decrement top pointer]

If stack is implemented using array it cannot grow dynamically where as Linked stack can grow dynamically.

6. What is doubly linked list (Dec 2016, April 2017,Nov 2017)

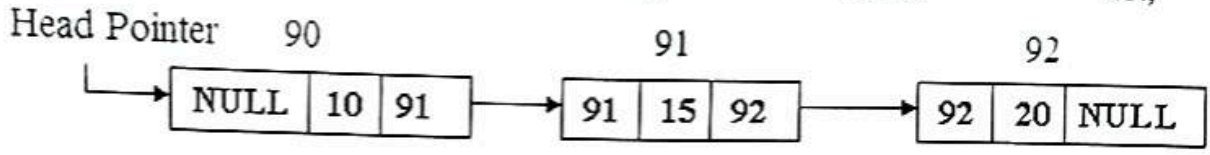
- Each node consist of three fields namely

  i.Previous address field or Backward link field,

ii.Data field or Information field,

iii.External address field or Forward link field.

- The previous address field holds the address of the previous node and the next address field holds the address of next node.
- In Double Linked list, we can move in both the direction from head pointer to Null address or vice versa.
- Consider the following linked list,



Operations of Double Linked list:

We can perform the two operations in double linked list:

i. Inserting
ii. Deleting

Inserting a node into the double linked list:

- If we want to insert a node into the double linked list, we need an empty node.
- Before inserting we should check whether the stack is underflow or not.
- If AVAIL=NULL, we can't get the empty node from the stack and we have to exit from the program.
- If AVAIL!= NULL, we can get the empty node from the stack .
- **Algorithm for inserting a node into the double linked list:**

DOUINS (L, R, X, M)

1. **[CHECK UNDERFLOW]**
   IF AVAIL= NULL THEN

   WRITE ("STACK UNDERFLOW")

   RETURN EXIT

2. **[OBTAIN THE ADDRESS OF THE NEW NODE]**
   NEW<- AVAIL

3. **[REMOVE THE NEW NODE FROM THE STACK]**
   AVAIL<- LINK (AVAIL)

4. **[FILL INFO FIELD OF NEW]**
   IF R=NULL THEN

LPTR (NEW) <- RPTR (NEW) <-NULL

L<-R<-NEW

5. [INSERT INTO EMPTY LIST]

IF R= NULL THEN

LPTR (NEW) <-RPTR (NEW)<-NULL

L<-R<-NEW

RETURN

6. [INSERTAS LEFT MOST NODE]

LPTR (NEW) <- NULL

RPTR (NEW) <-M

LPTR (M) <-NEW

L<-NEW

7. [INSERT IN MIDDLE]

LPTR (NEW) <-LPTR (M)

RPTR (NEW) <-M

LPTR (M) <-NEW

RPTR (LPTR (NEW)) <-NEW

7. How stack is useful to evaluate the Expression(Nov 2014,April 2017)

EXPRESSION

Usual algebraic notation is often termed infix notation. Here, the arithmetic operator appears between the two operands to which it is being applied. There are two alternate notations for expressing the expression. These are prefix and postfix expressions.

Example: A + B INFIX

+ A B PREFIX

A B + POSTFIX

The prefixes "pre -", "post -" and "in -" refer to the relative position of the operator with respect to the two operands. In prefix notation, the operator precedes the two operands, in postfix notation, the operator follows the two operands and in infix notation, the operator

is between the operands.

To understand the expression, we must first decide the order of evaluation of an expression. In C language, the operator with highest priority is evaluated first. Each operator is given certain priority.

Priority of arithmetic operators in C:

* / *High*

\+ - *Low*

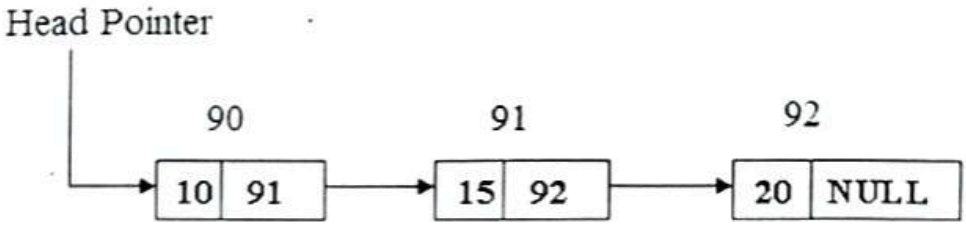
If the expression consists of more than one operator at the same level, then it follows left associativity rule. I.e., operator at left most will be done first and so evaluated from left to right.

To implement the conversion we make use of stack because it is easier to implement the concept.

8. Explain about single linked list in detail (Nov 2017)

Develop the algorithm for inserting a node as first node.

- In the single linked list each link has single link to the next node.
- It is otherwise called as linear linked list.
- It contains the head pointer which holds the address of the first node.
- Using head pointer only we can access entire linked list.
- The below diagram shows the single linked



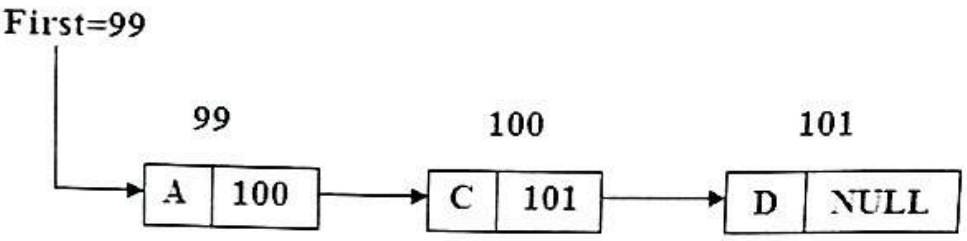
- In single linked list we can traverse in one direction from head to null.
- We cant move in reverse direction i.e. from null to head.
- Link field of last node have the null pointer indicates the end of the list.

Operations In Single Linked List:

- We can insert and delete the node in a single linked list.

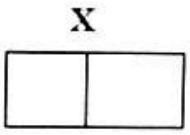
Inserting the Node into a Single Linked List:

Consider the linked list:



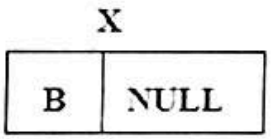
Where first is a pointer which holds the address of the first node.

If we want to insert a data B , into the list , first we need the empty node with empty data and link field.



Consider x is the address of new node.

Now put the data B into the data field of a new node and put NULL into the link field.

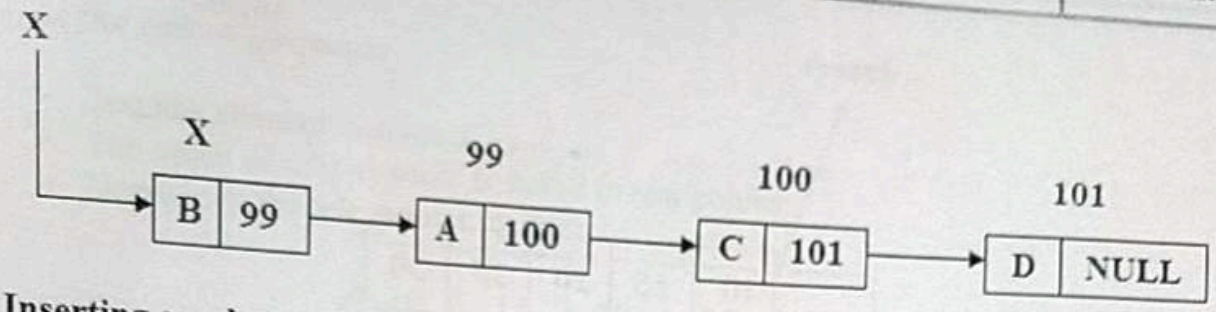


We can insert the new node by any one of the following position:

4. As a first node
5. As a intermediate node
6. As a last node.

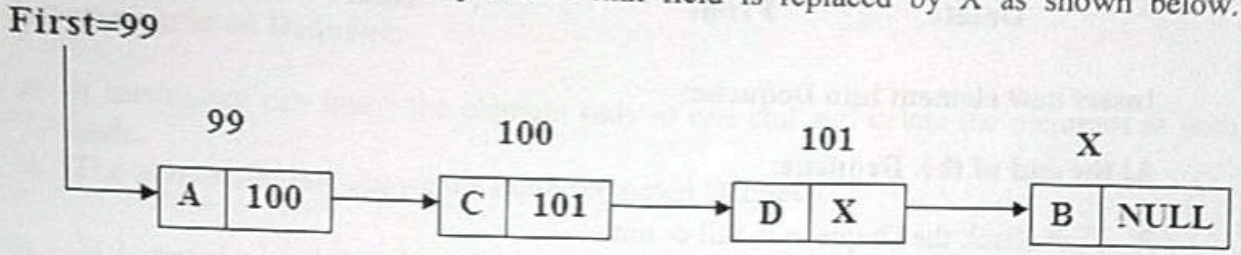
Inserting as a first node:

If we want to insert a new node as a first node, the link field of new node is replaced by FIRST and FISRT is replaced by X as shown below.



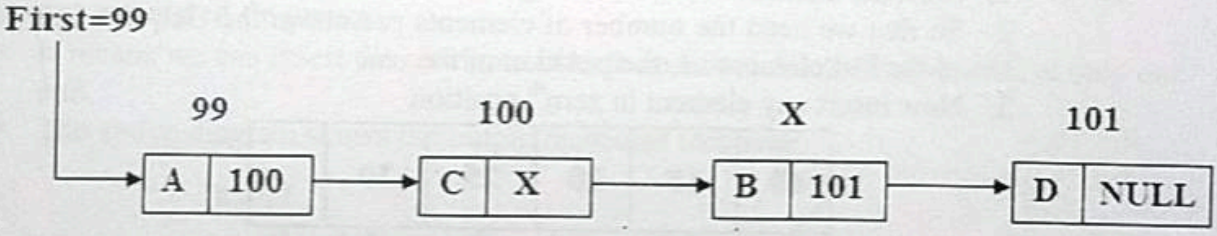
Inserting as a last node:

If we want to insert a node as a last node, the link field of new node is replaced by NULL and the link field of last but previous link field is replaced by X as shown below.



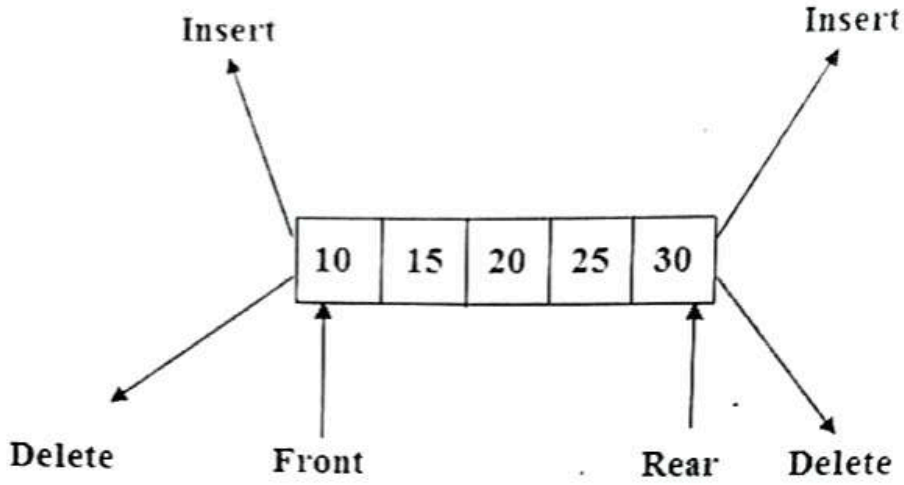
Inserting as an intermediate node:

- If we want to insert a new node as the intermediate node, first we need the address of the previous node.
- Now insert the new node in between the respective node as shown below
- If we want to insert a new node X as intermediate node, insert in between C and D.
- The link field of 99 is 100, the link field of 100 is 101 and the link field of 101 is 102.



9. State the operations performed on queue (ENQUEUE, DEQUEUE)(Nov 2017,Nov 2015)

- Dequeue is the short form of double ended queue
- In which we can insert or delete the elements either at front or rear
- The below diagram shows the double ended queue.



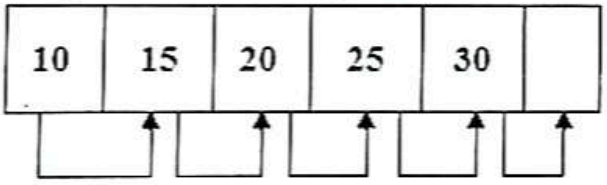
Insert new element into Dequeue:

At the end of the Dequeue:

- First check the Dequeue is full or not.
- We have to check whether the element is first to be added or inserted if it is true assign front and rear is 0.
- Now insert a element.

At the beginning of the Dequeue:

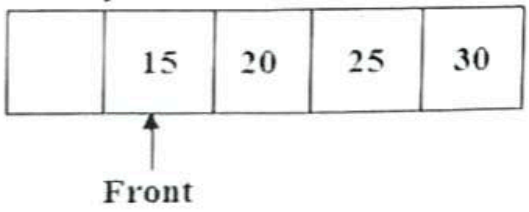
- First check the Dequeue is full or not.
- We have to check whether the element is first to be inserted if it is true then perform the following steps
    1. Shift the elements from left to right
    2. So that we need the number of elements present in the Dequeue ,the position of the last element i.e. the position of the rear
    3. Now insert any element in zeroth position



Deleting the element from Dequeue:

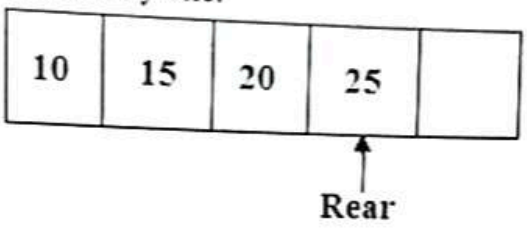
At the beginning of Dequeue:

- Delete the first element from the front position of the Dequeue
- The index of next element is stored in front pointer.
- Increment the front pointer by one.



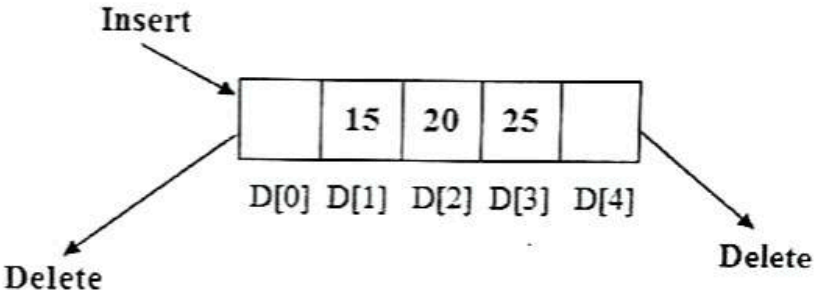
At the end of Dequeue:

- The last element is deleted.
- The index of next element is stored in rear pointer
- Decrement the rear pointer by one.



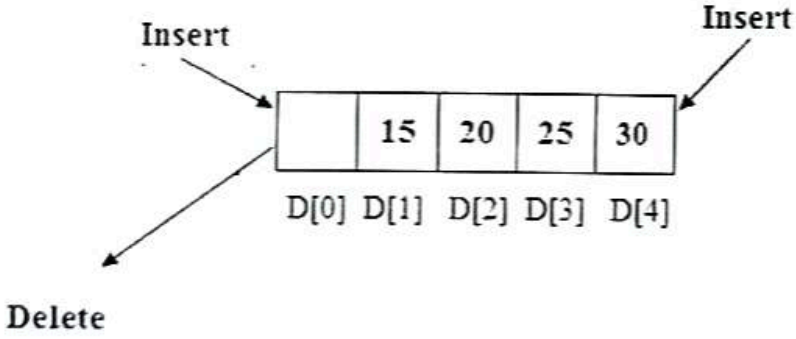
Input restricted Dequeue:

- It means we can insert the element only at one end and delete the elements at both ends.
- The above diagram shows the input restricted Dequeue.



Output restricted dequeues:

- It means we can insert the elements in both ends and delete the elements at only one end.
- The above diagram shows the output restricted Dequeue.



10. Explain about circular linked list (Nov 2015).

Circular linked list is a linked list where all nodes are connected to form a circle. There is no NULL at the end. A circular linked list can be a singly circular linked list or doubly circular linked list.

Advantages of Circular Linked Lists:

1) Any node can be a starting point. We can traverse the whole list by starting from any point. We just need to stop when the first visited node is visited again.

2) Useful for implementation of queue. Unlike this implementation, we don't need to maintain two pointers for front and rear if we use circular linked list. We can maintain a pointer to the last inserted node and front can always be obtained as next of last.

3) Circular lists are useful in applications to repeatedly go around the list. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.

4) Circular Doubly Linked Lists are used for implementation of advanced data structures like Fibonacci Heap.

11. Give an algorithm to add and delete elements of a circular queue in array representation.(Sep 2020):

- **Circular queue** is another form of a linear queue in which the last position is connected to the first position of the list.
- The circular queue is similar to linear queue has two ends, the front end and the rear end.
- The rear end is where we insert the elements and the front end is where we delete the elements.
- The traversing is in only one direction (i.e., from front to rear).
- Initially the front and rear ends are at same position(i.e., -1);
- While inserting elements the rear pointer moves one by one (where front pointer doesn't change) until the front end is reached.
- If the next position of the rear is front, the queue is said to be fully occupied.
- Beyond this insertion is not done.
- But if we delete any data, we can insert the element accordingly.
- While deleting the elements the front pointer moves one by one (where as the rear point doesn't change) until the rear point is reached.

- If the front pointer reaches the rear pointer, both the ir positions are initialized to -1, and the queue is said to be empty.
- A more efficient queue representation is obtained by the circular array.
- It becomes more convenient to declare the array as Q[n-1].
- When rear = n-1, the next element is entered at Q[0] in case that position is free.
- Using the same conventions, front will always point one position counterclockwise from the first element in the queue.
- Again, *front=rear* if and only if the queue is empty. Initially we have *front = rear =* 1.
- The following figure illustrates some of the possible configurations for a circular queue containing the four elements with n>4.

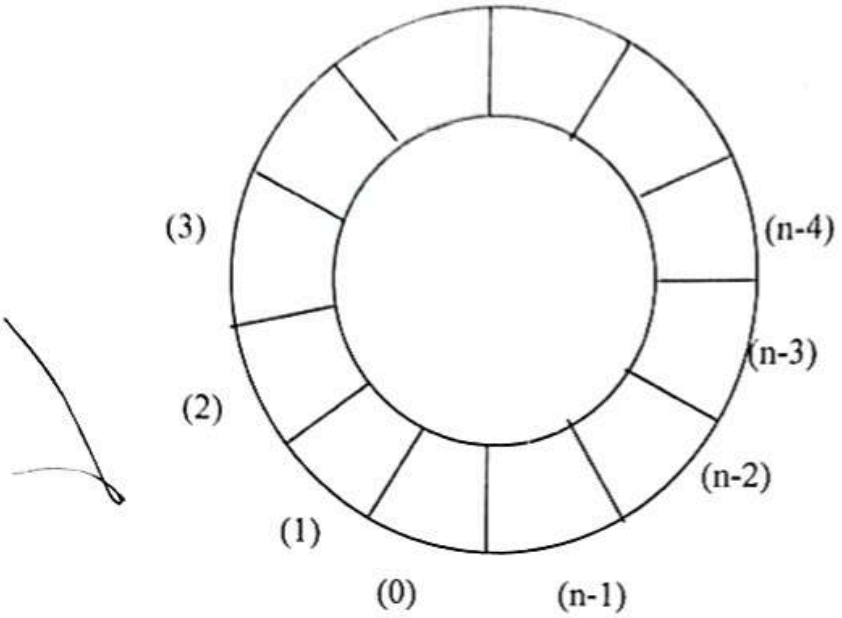
❖ *front = 0 ;rear=4*

- The assumption of circularity changes the ADD and DELETE algorithm slightly.
- In order to add an element , it will be necessary to move *rear* one position clockwise, i.e.,

if *rear* = n-1 **then** *rear* =0

else *rear* = *rear*+1.

- Using modulo operator which computes remainders, this is just *rear*=(*rear*+1)**mod n.**
- Similarly, it will be necessary to move **front** one position clockwise each time a deletion is made.
- Again, using the modulo operation, this can be accomplished by *front=(front+1)***mod** *n.*
- An examination of the algorithms indicates that addition and deletion can now be carried out in a fixed amount of time or ***O*(1).**



- **front = n-4 , rear = 0**

Procedure *ADDQ* (*item, Q, n, front, rear*)

//insert item in the circular queue stored in Q (0: n-1);

rear points to the last item and *front* is one position counter clockwise from the first item in Q//

rear = (*rear*+1)**mod** *n* //advance rear clockwise//

if *front* = *rear* **then call** QUEUE-FULL

Q(*rear*)= item //insert new item //

end *ADDQ*.

Procedure *DELETEQ* (*item, Q, n, front, rear*)

//removes the front element of the queue Q (0: n-1)//

if *front* = *rear* **then call** QUEUE-EMPTY

front = (*front* + 1)**mod** *n* //advance front clockwise//

item = Q(*front*) //set item to front of queue//

end *DELETEQ*